



SDA

STATE CHART DIAGRAMS

Syed Ahmed Khan
`syed.ahmed@nu.edu.pk`

LECTURE OUTLINE

- Understand what State Chart Diagrams are.
- State Chart Diagrams core elements and notations.
- State Chart Diagrams purpose.
- Composite and Concurrent states.
- Examine Transitions and the Events that trigger state changes

STATE CHART DIAGRAMS

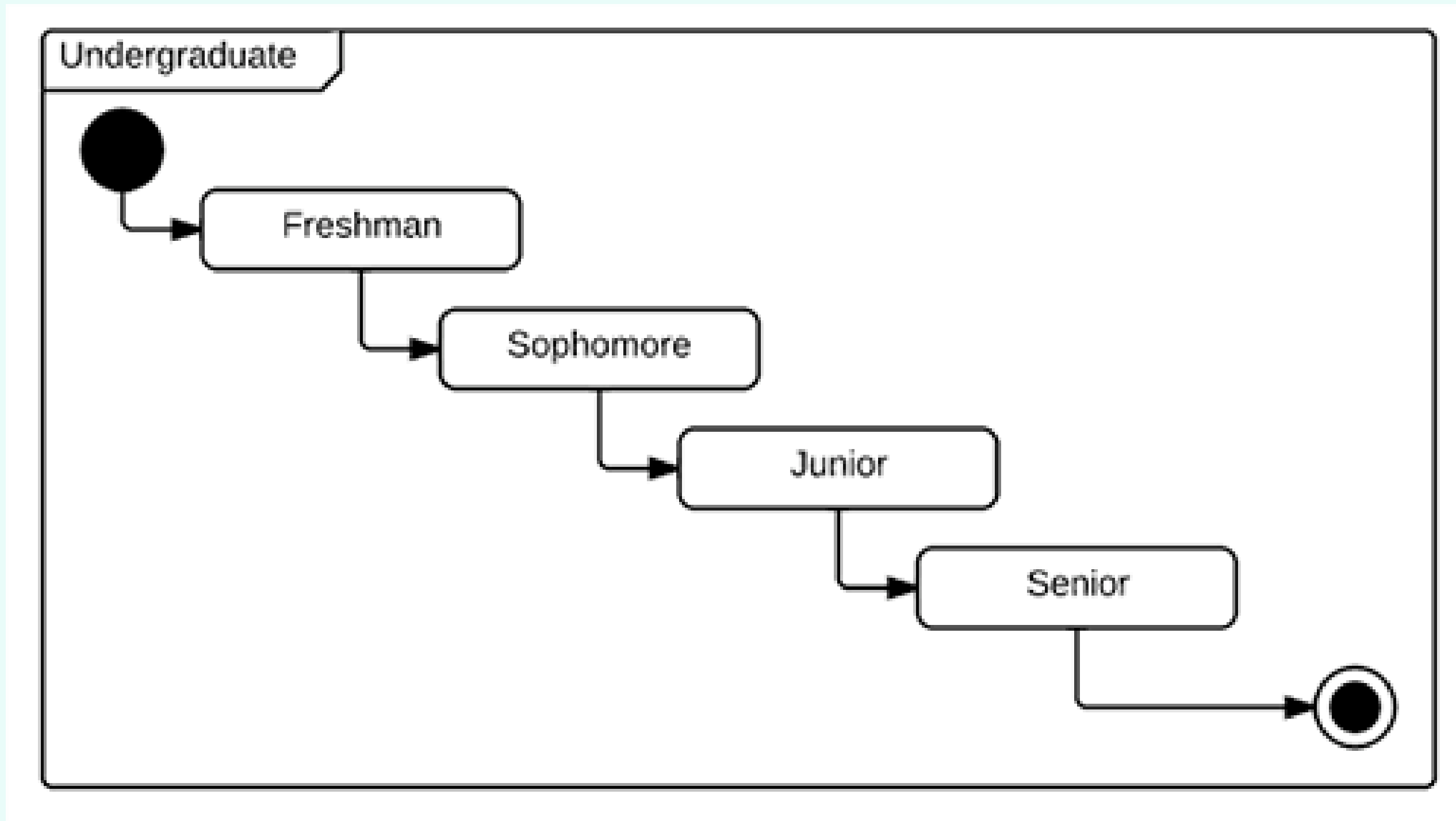
A State Chart Diagram (or State Machine Diagram) models the lifecycle of an object, showing the states it can be in and how it transitions between those states due to events or actions.

Example: An ATM card can be in states like Inserted, Verified, Ejected, or Blocked. The card's state changes only when some event occurs, such as the user entering a PIN or removing the card.

STATE CHART DIAGRAMS

State charts are excellent for showing how an object's behavior changes across multiple use case executions. However, they are not good for showing interactions between multiple objects. For that, you'd use a Sequence Diagram.

STATE CHART DIAGRAMS



STATE CHART DIAGRAMS

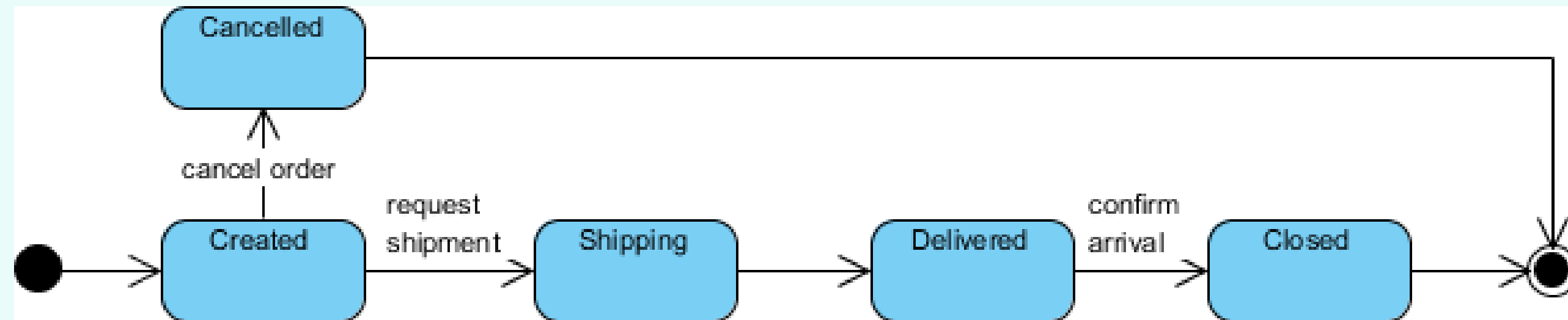
Another Example: Library Book object.

Possible states might be:

Available → Reserved → Borrowed → Returned → Available
again.

Transitions are triggered by user actions – reserving, borrowing,
or returning the book.

ORDER STATE CHART DIAGRAMS



WHY USE STATE CHART DIAGRAMS?

The power of state charts is that they model state-dependent behavior. This means an object can respond differently to the same event depending on what state it's in.

For example, the event `keyPress` will have a different effect if your keyboard is in the `"caps_locked"` state versus the `"default"` state. We usually apply this to objects, but it can be used for any element with behavior, like a whole system or a use case.

WHY USE STATE CHART DIAGRAMS?

state charts are often used hand-in-hand with Sequence Diagrams. A state chart describes all possible events for a single object; a sequence diagram describes the events for a single interaction across all involved objects.

WHY USE STATE CHART DIAGRAMS?

state charts are often used hand-in-hand with Sequence Diagrams. A state chart describes all possible events for a single object; a sequence diagram describes the events for a single interaction across all involved objects.

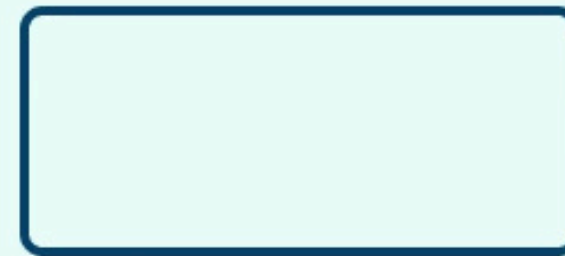
ELEMENTS OF STATE CHART DIAGRAMS

The three core elements of a State Chart Diagram are;

- State
- Transition
- Events

STATE

A State is a condition or situation during the life of an object where it satisfies some condition, performs some activity, or waits for an event. Examples: an Invoice is Paid. A Car is Standing Still. Jim is Acting as a Salesman

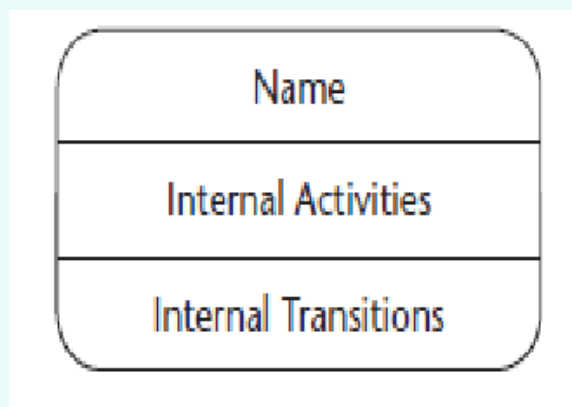
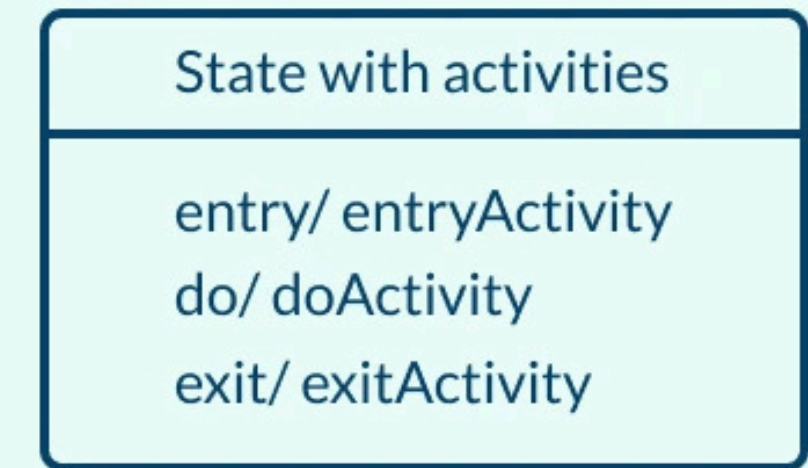


State

STATE

A state can have up to three compartments.

1. The Name of the state (e.g., Idle, Processing).
2. The optional Activity compartment. This lists behaviors in response to events. UML has three standard events here:
 - entry: An action performed upon entering the state (e.g., entry / assignValue).
 - exit: An action performed upon exiting the state.
 - do: A long-running activity performed while inside the state (e.g., do / runSelfTest).
3. The optional Internal Transitions compartment. These are transitions that cause an action but do not result in a change of state.



INITIAL AND FINAL STATE

Two special states are crucial:

- The Initial State is where the object's life begins. It's a single, filled black circle.
- The Final State is where the object's life ends. It's a filled circle inside a larger circle. An object can have multiple final states.

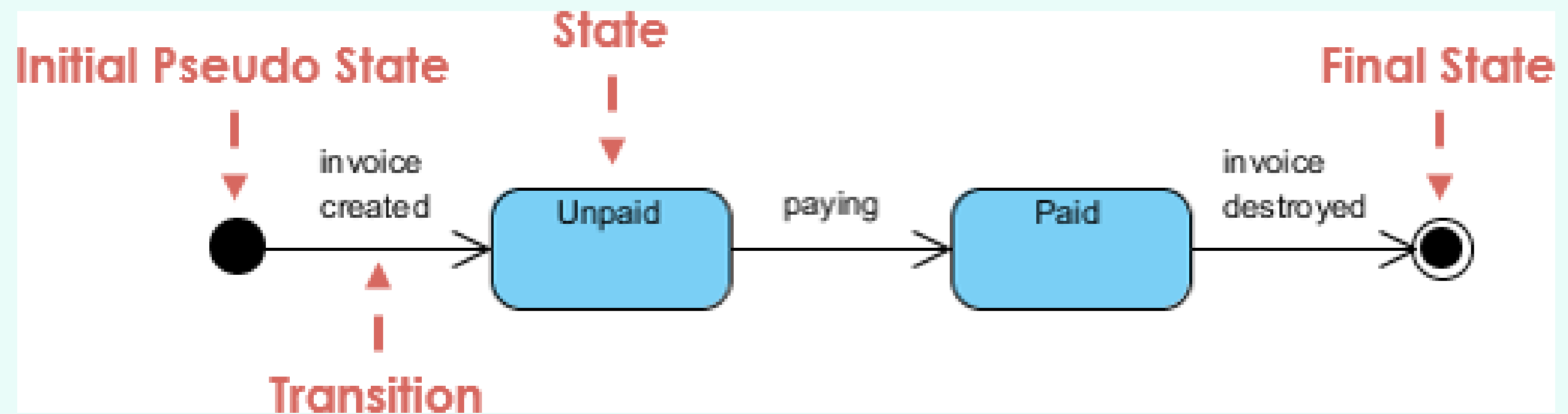


Initial State



Final state

INITIAL AND FINAL STATE



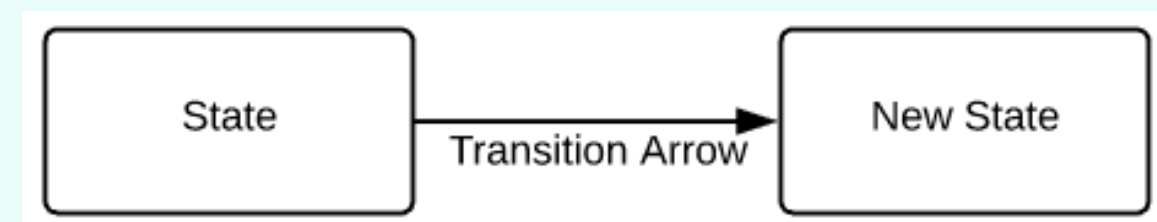
TRANSITION

A **transition** is a solid arrow from a source state to a target state.
It can be labeled with the following syntax:

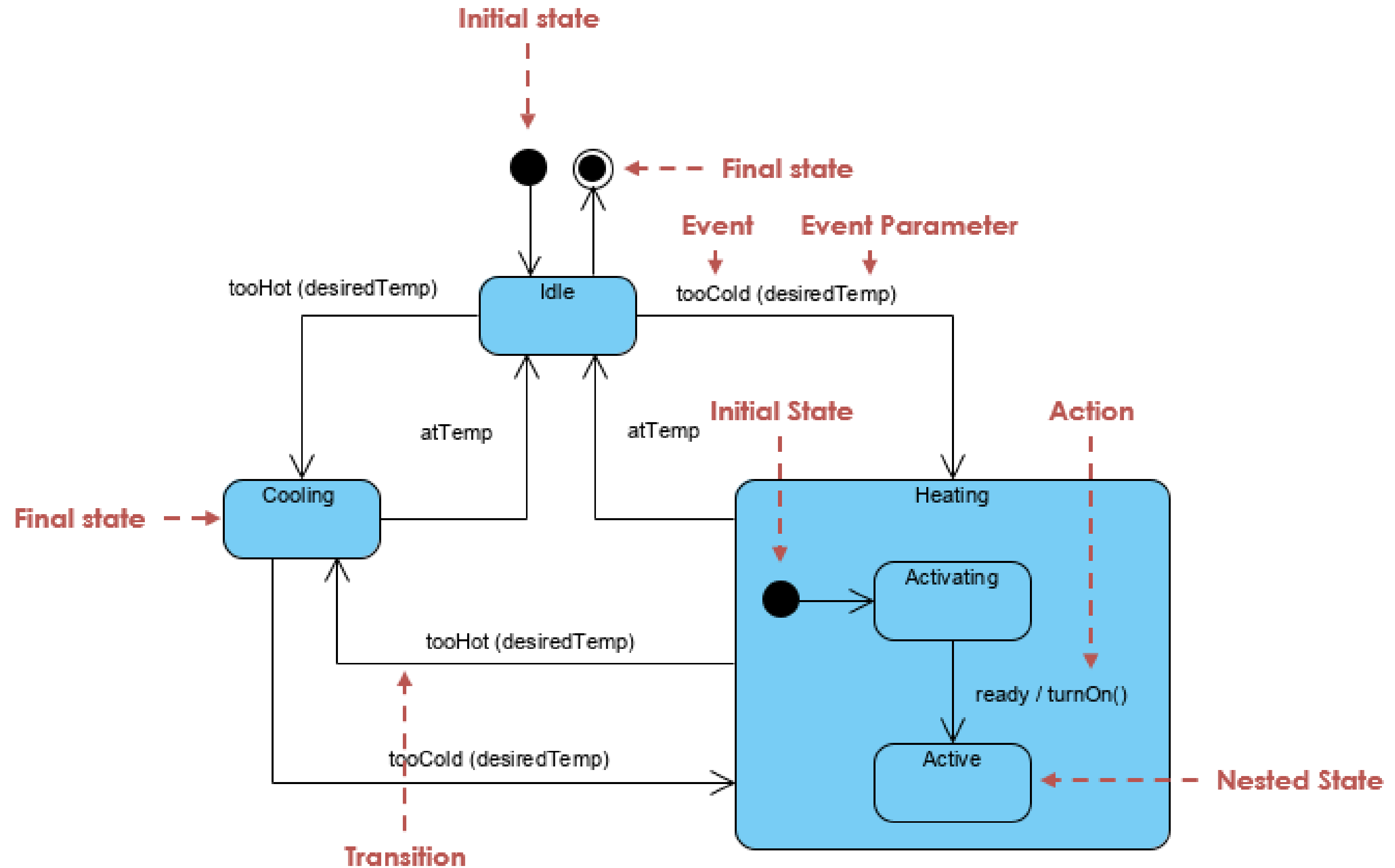
Event [Guard] / Action



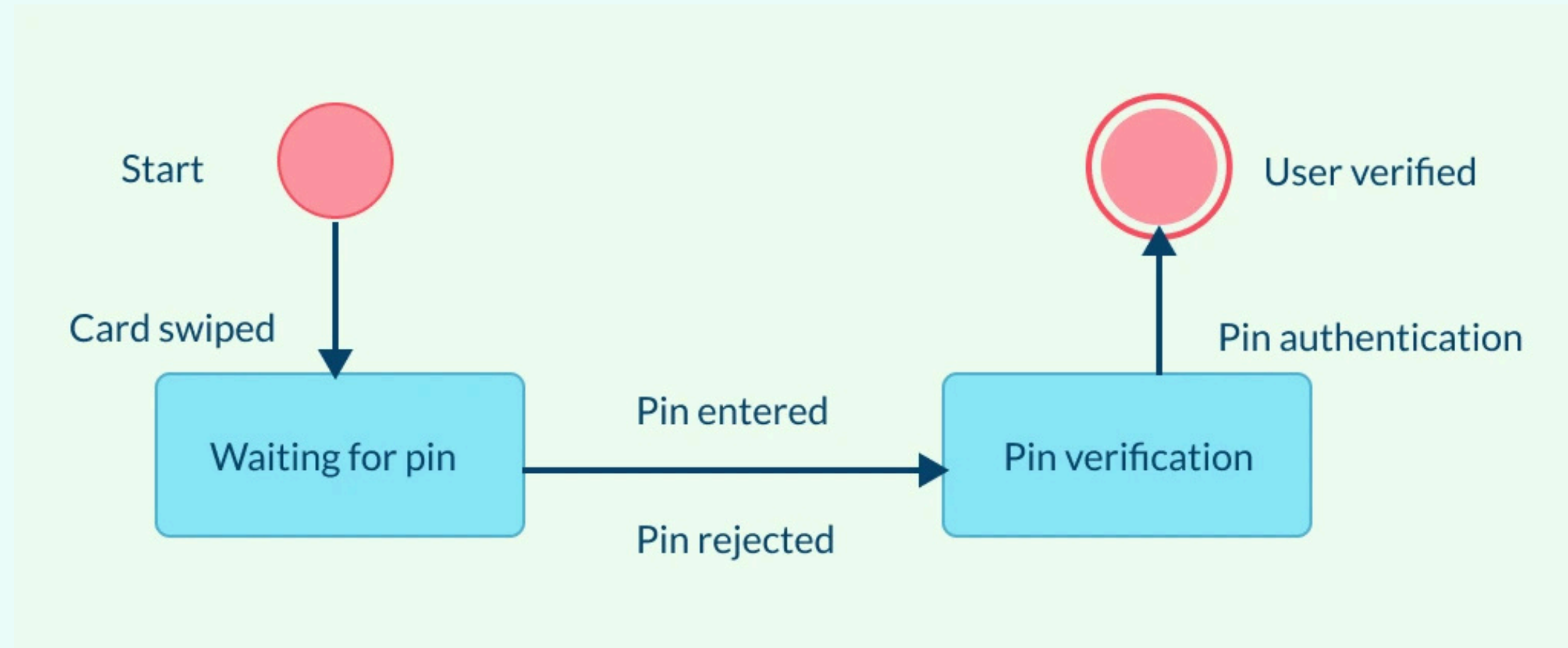
- Event: The happening that triggers the transition.
- Guard: A Boolean condition in square brackets [] that must be true for the transition to fire.
- Action: An executable, atomic computation that occurs during the transition.



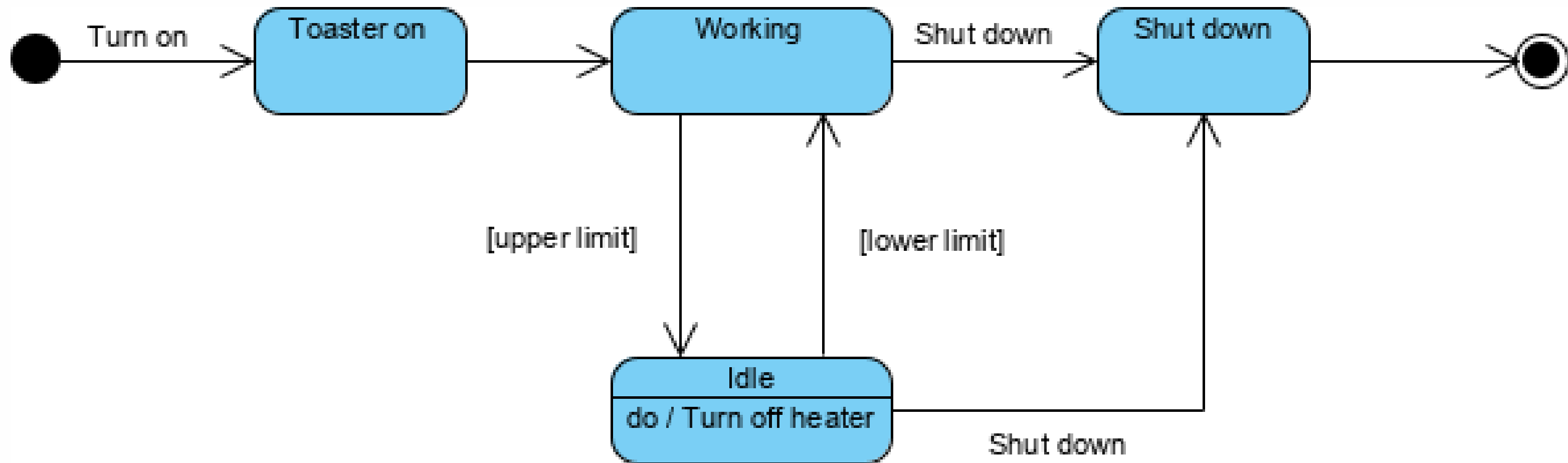
A STATE CHART DIAGRAM AT A GLANCE



EXAMPLE 1



EXAMPLE 2



EVENTS

An object transitions (changes) from one state to another state when something happens, which is called an event.

UML defines four primary types of events:

1. **A Change Event:** When a Boolean condition becomes true. Written as `when(condition)` or shown as a guard `[condition]`.
2. **A Signal Event:** The receipt of an explicit signal from another object. This is a message.
3. **A Call Event:** The receipt of a call for an operation from another object. This is also a message.
4. **A Time Event:** The passage of a designated period of time. Written as `after(5 seconds)` or `when(date)`.

EVENTS

An event is an instant in time that is significant to the object.
Events are written as simple text.

- Open
- Close
- Withdraw
- Deposit
- Submit

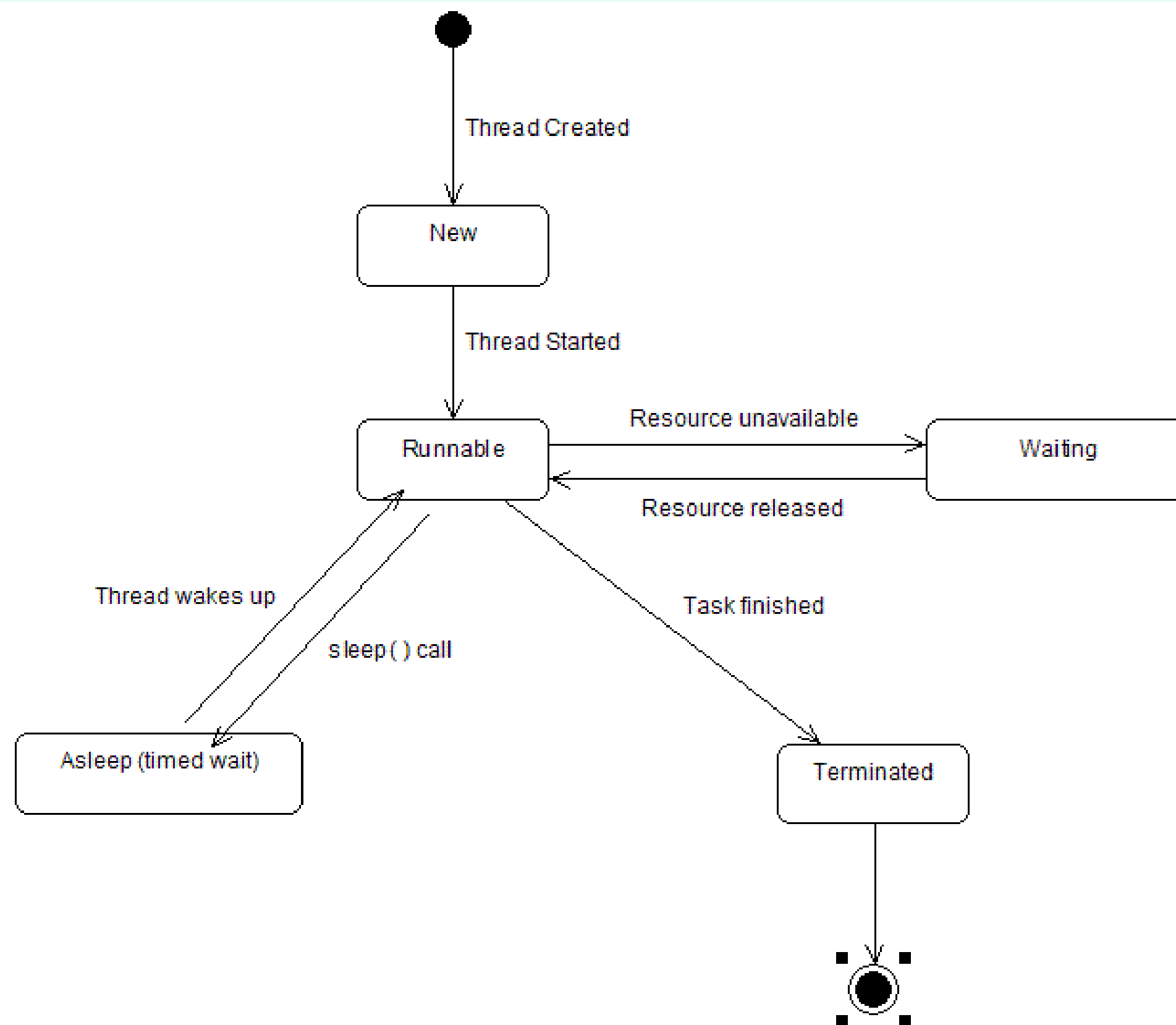
GUARD CONDITION

The Guard Condition is a powerful concept. It's a Boolean expression that must be true for a transition to be taken. For example, **withdraw(amount) [balance >= amount]**. The event is withdraw, but the transition only happens if the guard **[balance >= amount]** is true. time related guards: **[t = 15sec]**.

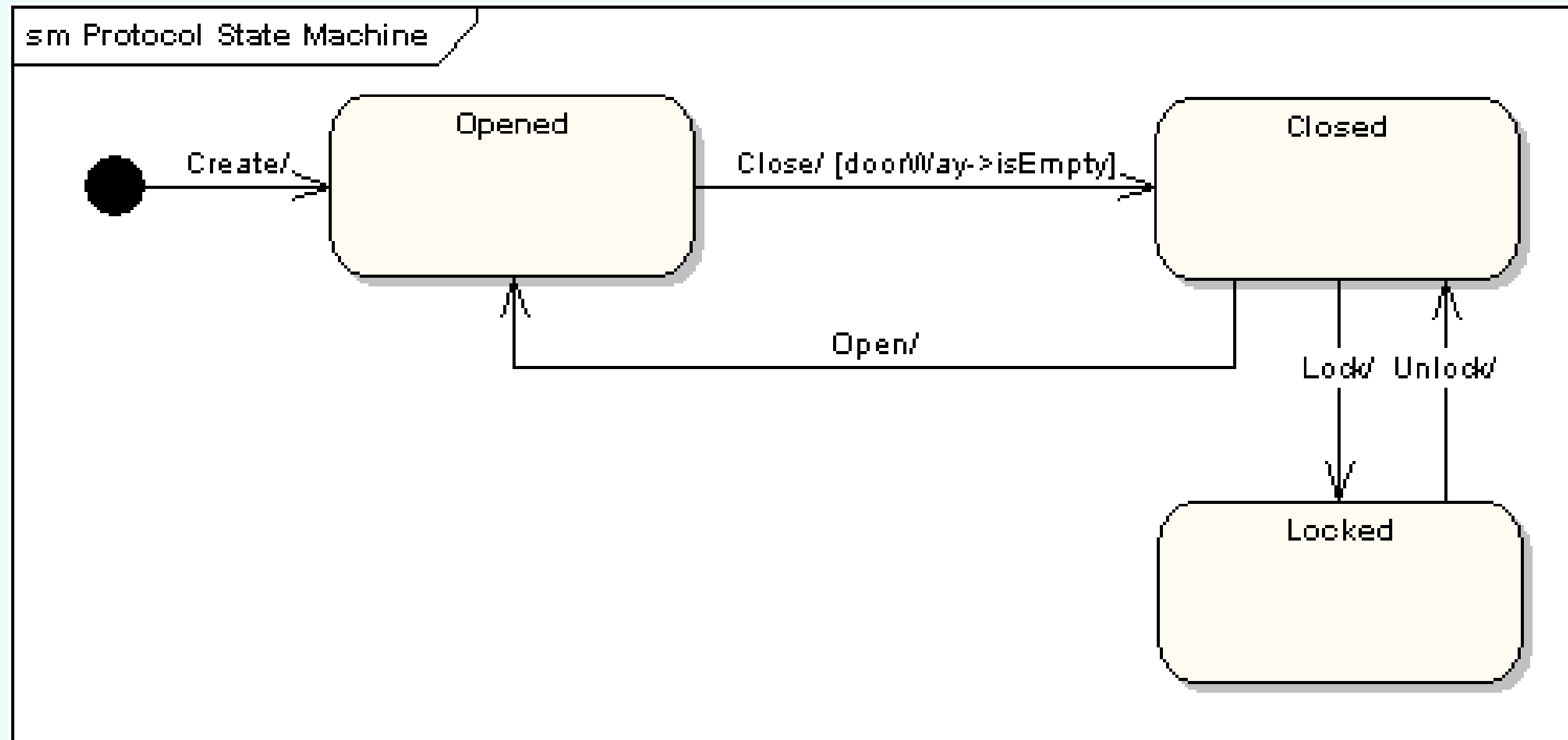
INTERNAL TRANSITION

This keyboard example perfectly illustrates internal transitions. The event **CAPS_LOCK** causes a state change from default to caps_locked. However, the event **ANY_KEY** is an internal transition. In the default state, It triggers the action **send_lower_case_scan_code()**, but the keyboard remains in the default state. No entry or exit actions are executed.

EXAMPLE

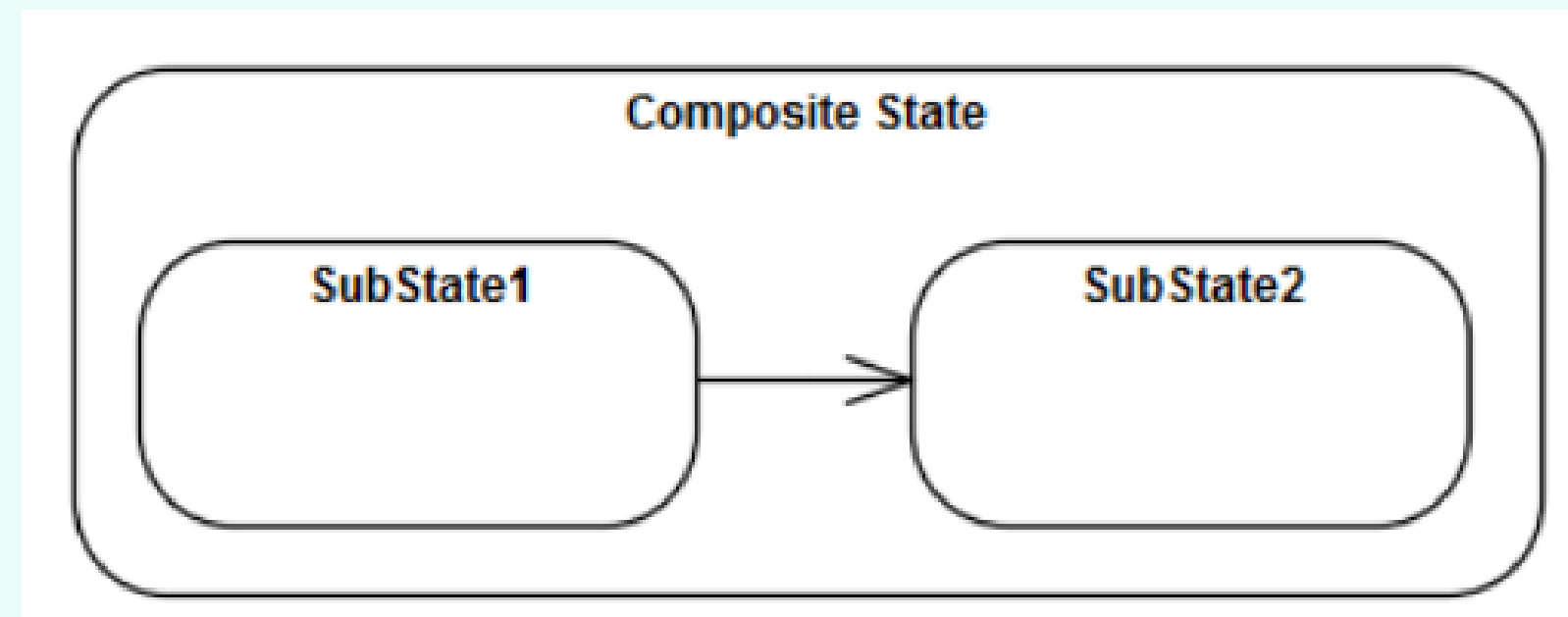


EXAMPLE

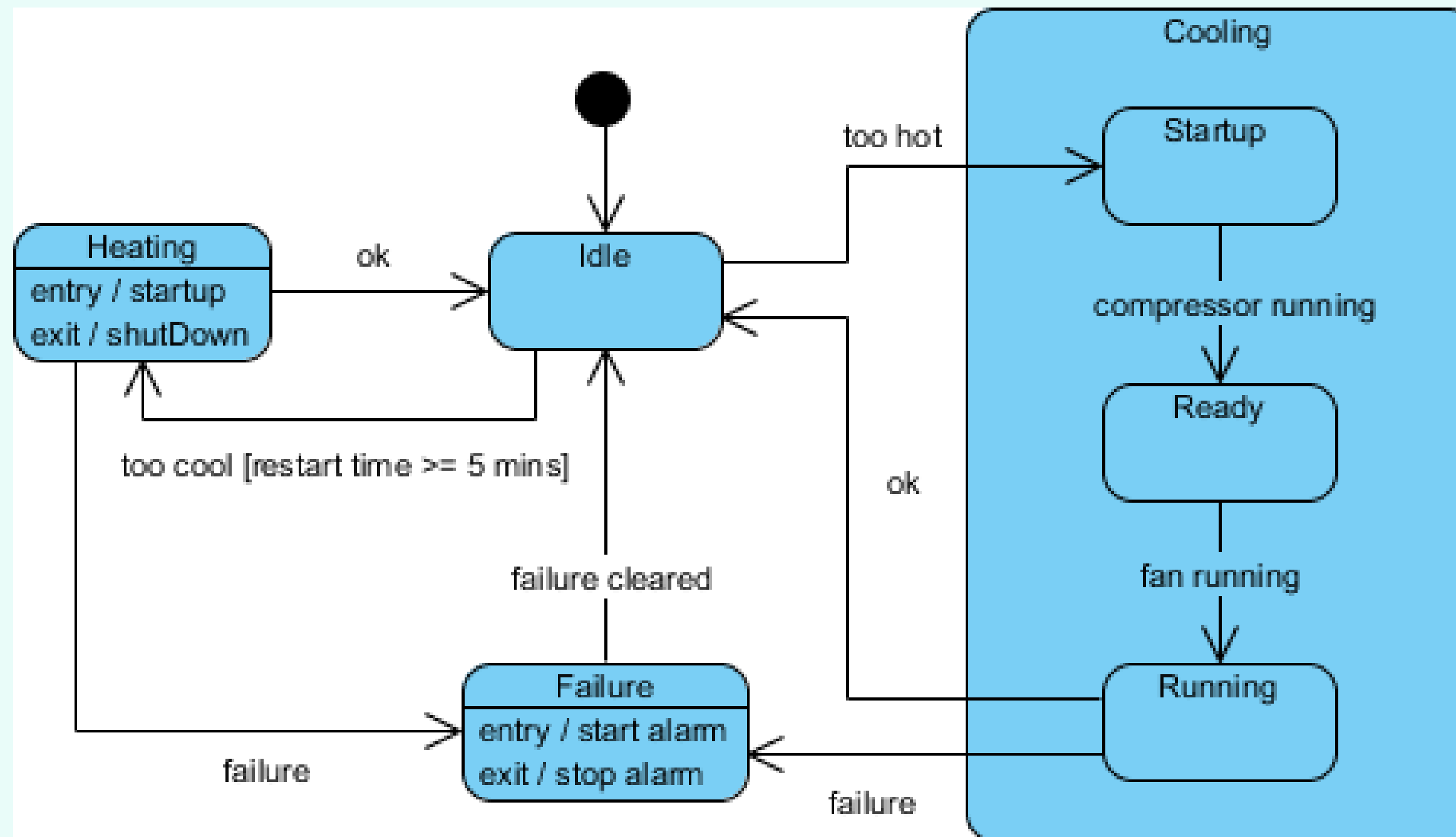


COMPOSITE STATE

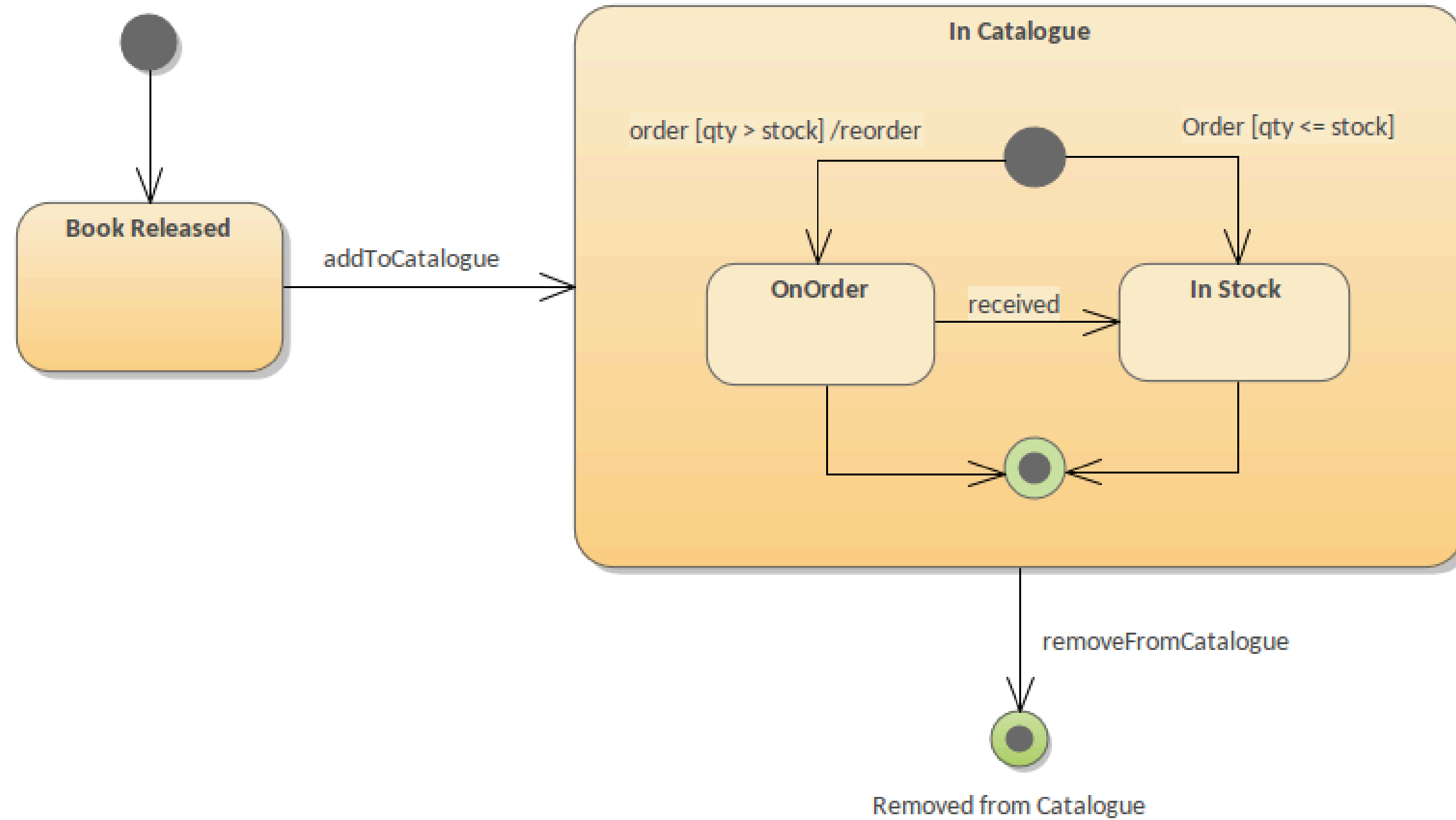
States can be nested. A Composite State is a state that contains substates. This is great for managing complexity. The key rule is: a transition into a composite state enters its initial substate. A transition out of a composite state applies to all substates inside it.



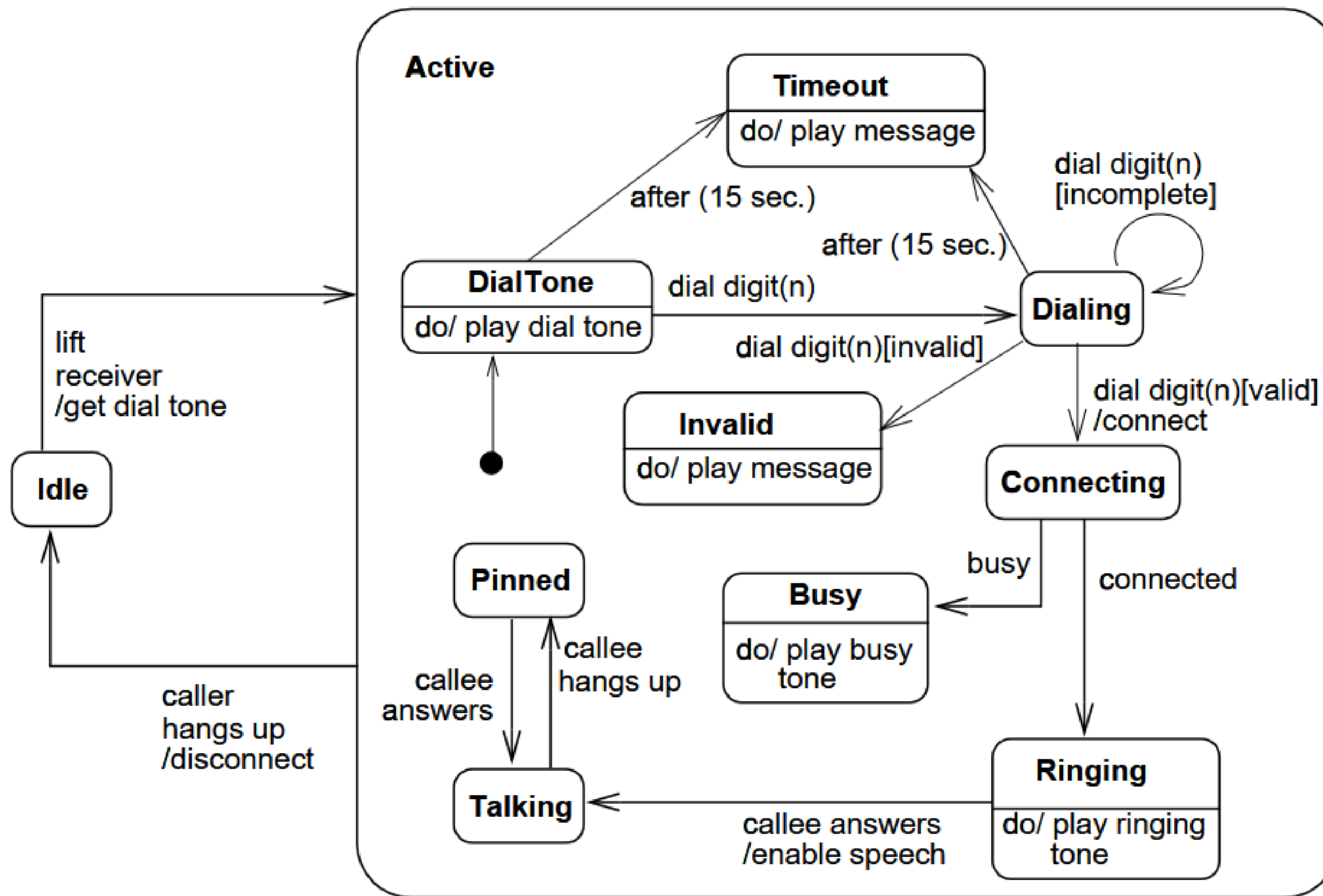
COMPOSITE STATE: HEATER EXAMPLE



COMPOSITE STATE



PHONE CALL: EXAMPLE

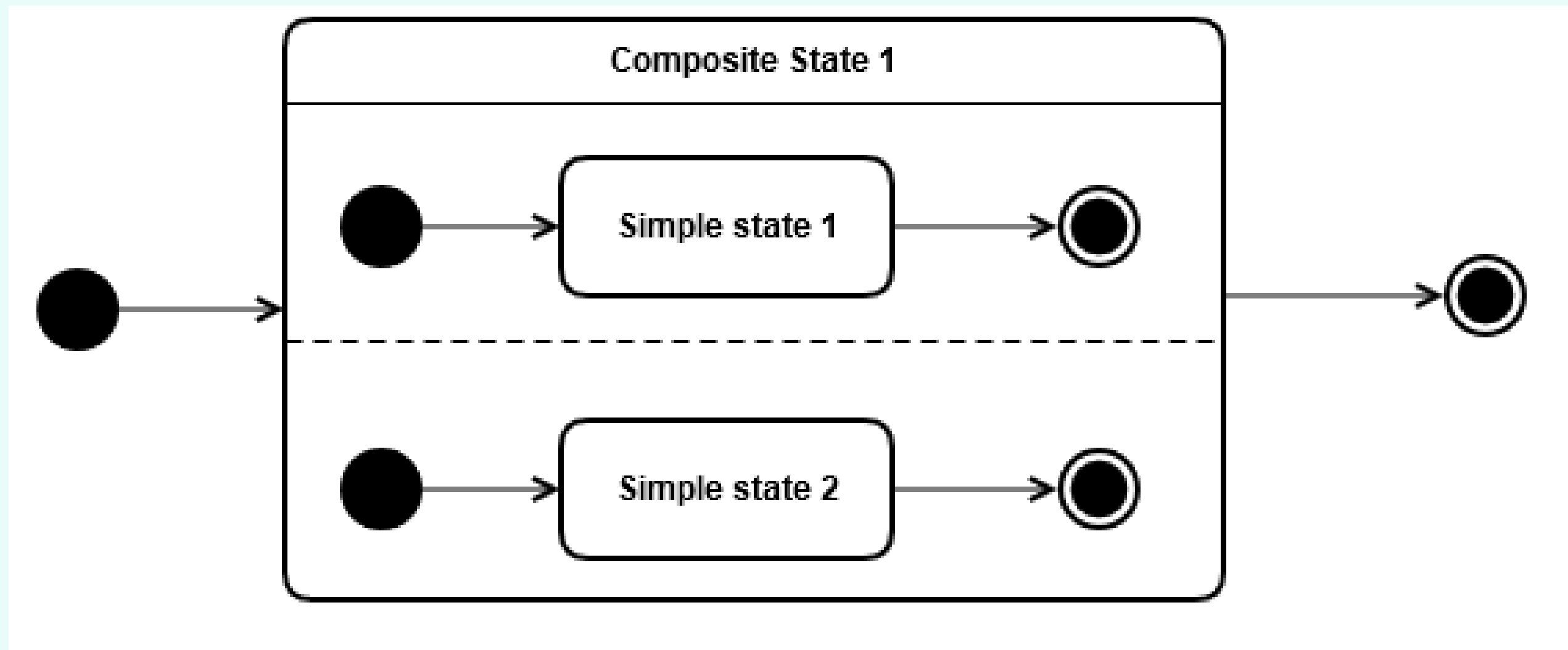


CONCURRENT STATE

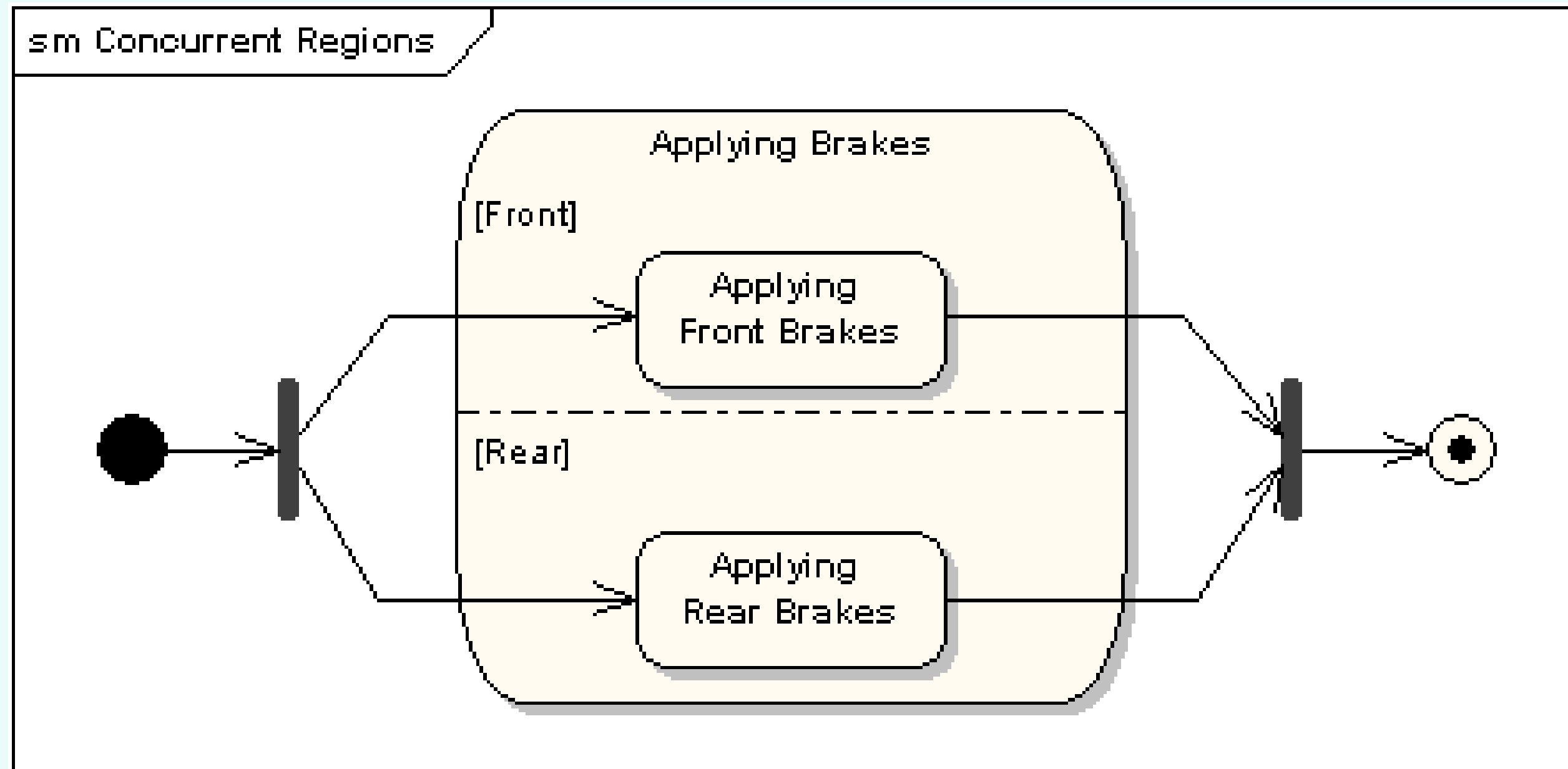
You can model an object being in multiple states at the same time by using an orthogonal state—a composite state divided into multiple regions by a dashed line.

For example: the state "Applying Brakes" might have two concurrent regions: "Front Brakes" and "Rear Brakes," each with their own substates. When you enter "Applying Brakes," you enter both regions simultaneously. You can also show this explicitly using fork and join pseudostates.

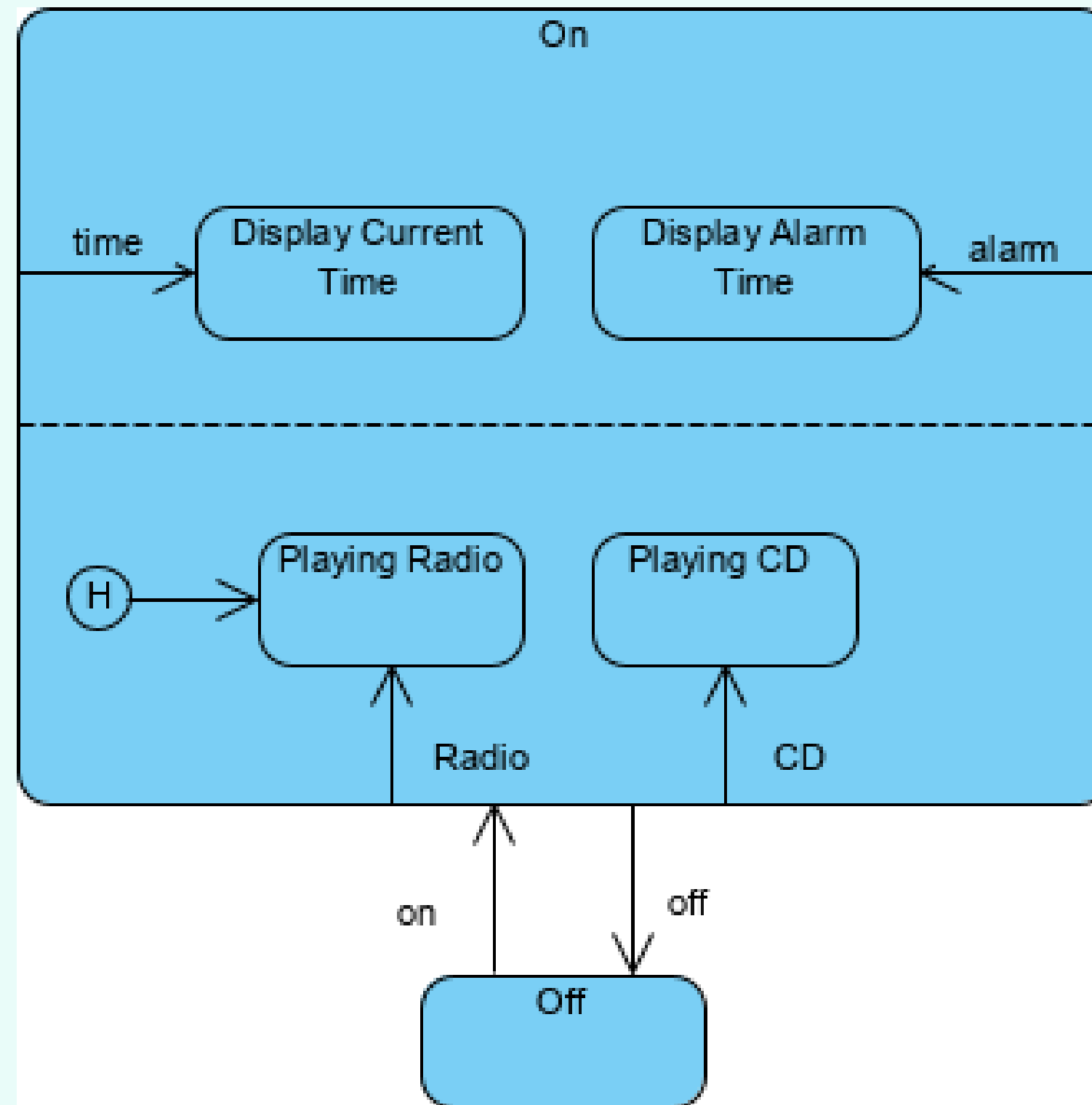
CONCURRENT STATE



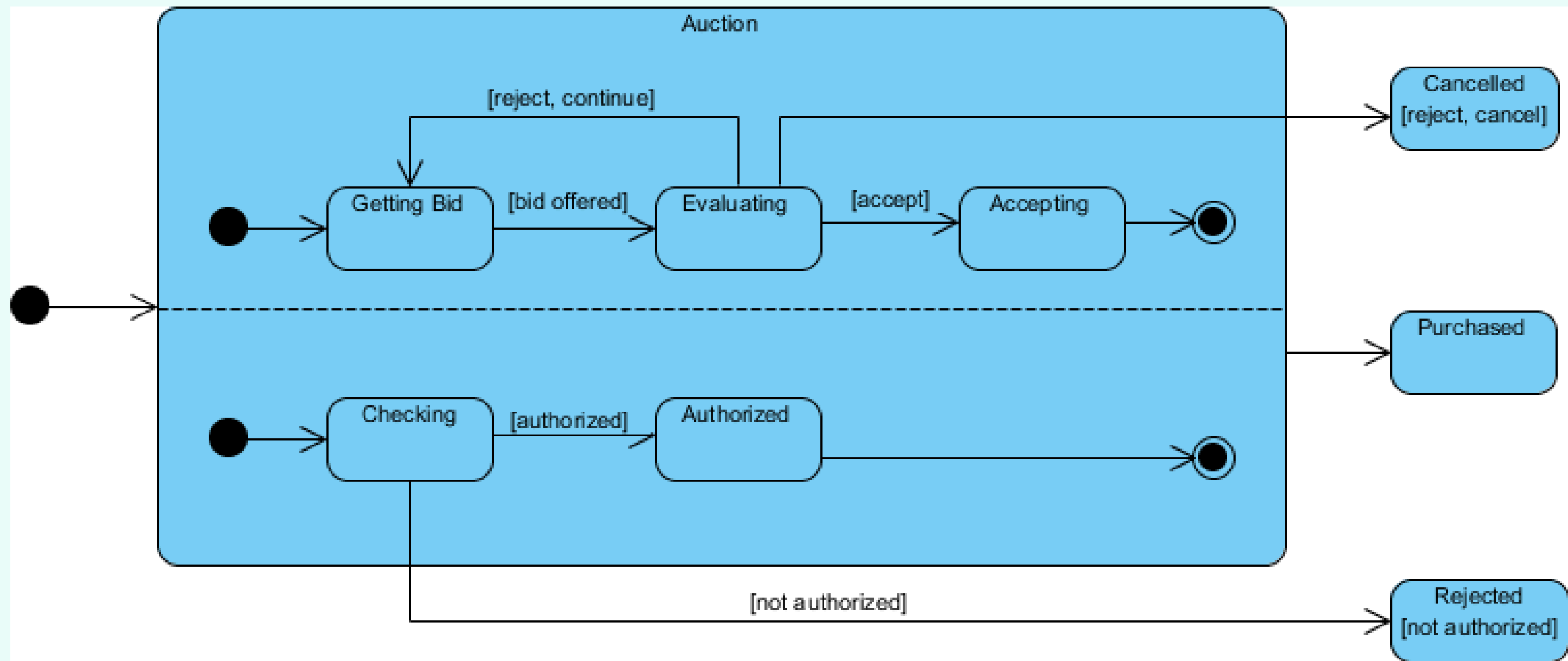
CONCURRENT STATE



CONCURRENT STATE



CONCURRENT STATE

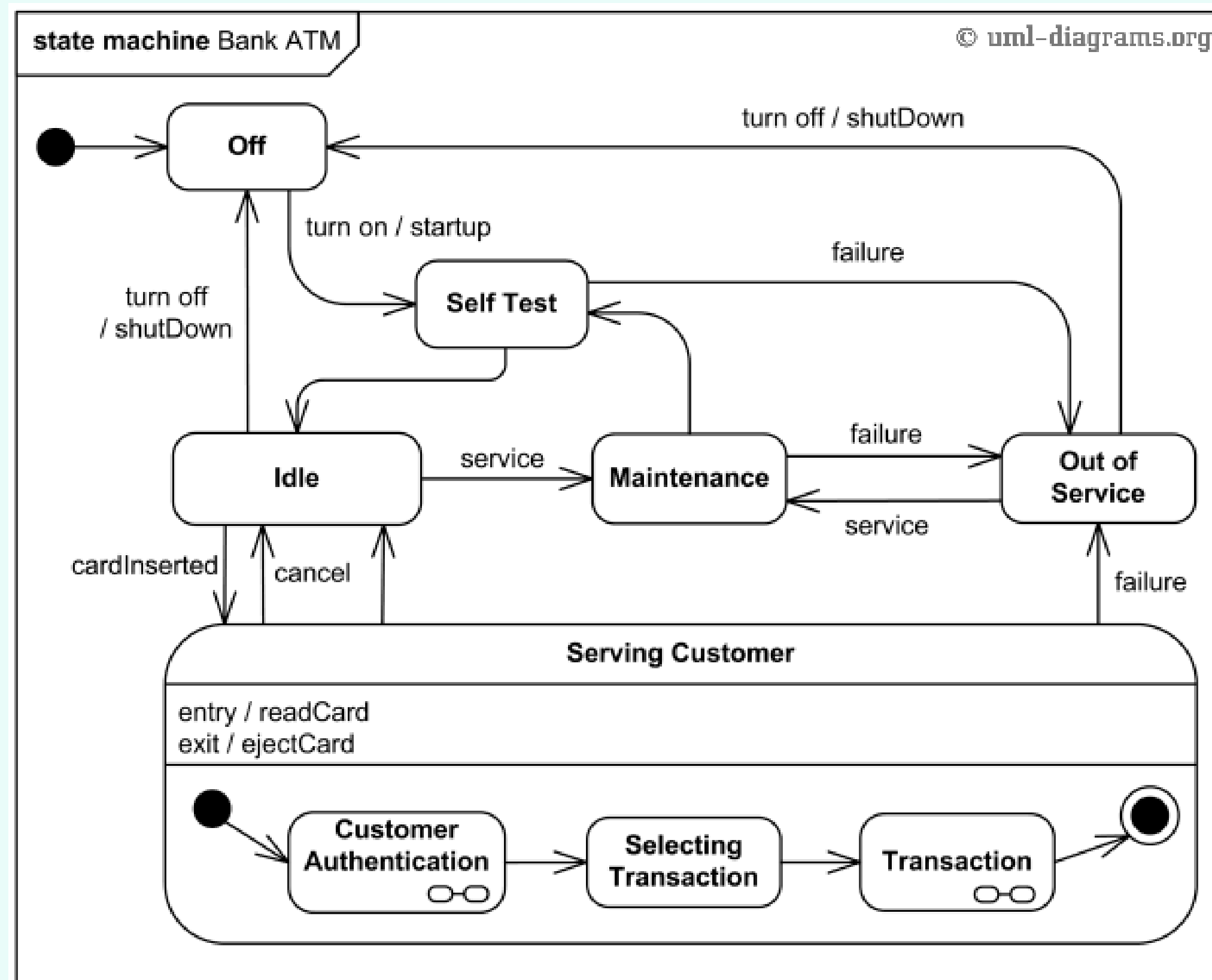


SCENARIO

ATM is initially turned off. After the power is turned on, ATM performs startup action and enters Self Test state. If the test fails, ATM goes into Out of Service state, otherwise there is triggerless transition to the Idle state. In this state ATM waits for customer interaction.

The ATM state changes from Idle to Serving Customer when the customer inserts banking or credit card in the ATM's card reader. On entering the Serving Customer state, the entry action readCard is performed. Note, that transition from Serving Customer state back to the Idle state could be triggered by cancel event as the customer could cancel transaction at any time.

SCENARIO





THANK YOU